

LAB 03

SOFTWARE: The image was converted to grayscale, and the matrix of the grayscale image was obtained in a .txt format using Python.



Original Image



Grayscale Image

```

  ▾ Convert image to grayscale matrix

0s ✓ ▶ import cv2
import numpy as np

# Upload the image file to Google Colab's runtime environment

# Load the uploaded image
image = cv2.imread('/content/HSCD_Lab3.jpg')

# Convert the image to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Convert the grayscale image to an integer matrix
integer_matrix = gray_image.astype(int)

# Save the integer matrix to a text file
np.savetxt('/content/grayscale_matrix.txt', integer_matrix, fmt='%d')

print("Integer matrix saved to grayscale_matrix.txt")

Integer matrix saved to grayscale_matrix.txt

  ▾ Displaying the grayscale image

0s ✓ ▶ import numpy as np
import cv2
from google.colab.patches import cv2_imshow

# Read the integer matrix from the text file
integer_matrix = np.loadtxt('/content/grayscale_matrix.txt', dtype=int)

# Generate the grayscale image from the integer matrix
gray_image = integer_matrix.astype(np.uint8)

# Display the grayscale image
cv2_imshow(gray_image)
```



HARDWARE: The dimension of the grayscale matrix obtained was 100x88, which was then read in a Verilog code. The Gx and Gy matrices' dimensions were obtained from the formula $n-k+1$, where n and k are the dimensions of the grayscale matrix and Sobel Kernels respectively. On performing the convolution for Gx and Gy and applying the Sobel Filter, we generated the final matrix.

Design:

```
design.sv  grayscale_matrix.txt
1 module sobel_filter();
2   integer fd,code,i,j,k,l;
3   reg [31:0] MEM[0:99][0:87];
4   reg [31:0] G1[0:2][0:2];
5   reg [31:0] G2[0:2][0:2];
6   reg signed[31:0] Gx[0:97][0:85];
7   reg signed[31:0] Gy[0:97][0:85];
8   reg signed[31:0] G[0:97][0:85];
9   reg [31:0] s;
10  initial
11  begin
12      //G1
13      G1[0][0] = 1;
14      G1[0][1] = 0;
15      G1[0][2] = -1;
16
17      G1[1][0] = 2;
18      G1[1][1] = 0;
19      G1[1][2] = -2;
20
21      G1[2][0] = 1;
22      G1[2][1] = 0;
23      G1[2][2] = -1;
24
25      //G2
26      G2[0][0] = 1;
27      G2[0][1] = 2;
28      G2[0][2] = 1;
29
30      G2[1][0] = 0;
31      G2[1][1] = 0;
32      G2[1][2] = 0;
33
34      G2[2][0] = -1;
35      G2[2][1] = -2;
36      G2[2][2] = -1;
37
38      //reading the grayscale matrix text
39      fd = $fopen("grayscale_matrix.txt", "r");
40      for(i=0; i<100; i++)
41      begin
42          for(j=0; j<88; j++)
43          begin
44              code = $fscanf(fd, "%d", s);
```

```
design.sv  grayscale_matrix.txt
39      fd = $fopen("grayscale_matrix.txt", "r");
40      for(i=0; i<100; i++)
41      begin
42          for(j=0; j<88; j++)
43          begin
44              code = $fscanf(fd, "%d", s);
45              MEM[i][j] = s;
46          end
47      end
48
49      //Convolution for Gx & Gy
50      for(i=0; i<98; i=i+1)
51      begin
52          for(j=0; j<86; j=j+1)
53          begin
54              Gx[i][j] = 0;
55              Gy[i][j] = 0;
56              for(k=0; k<3; k=k+1)
57              begin
58                  for(l=0; l<3; l=l+1)
59                  begin
60                      Gx[i][j] = Gx[i][j] + (MEM[k+i][l+j]*G1[k][l]);
61                      Gy[i][j] = Gy[i][j] + (MEM[k+i][l+j]*G2[k][l]);
62                  end
63              end
64          end
65      end
66
67      //Sobel Matrix G
68      for(i=0; i<98; i=i+1)
69      begin
70          for(j=0; j<86; j=j+1)
71          begin
72              G[i][j] = $sqrt((Gx[i][j]*Gx[i][j]) + (Gy[i][j]*Gy[i][j]));
73          end
74      end
75
76      //Display Sobel Matrix
77      for(i=0; i<98; i=i+1)
78      begin
79          for(j=0; j<86; j=j+1)
80          begin
81              $write("%d ",G[i][j]);
82              $display(" ");
83          end
84      end
85  endmodule
```

SOFTWARE:

The filtered matrix was stored in a .txt file which was converted to an image using Python. The image on applying Sobel filter has been converted to an image that emphasizes edges.

